To move canvas, hold Scroll wheel or Space while dragging, or use the hand tool

Questão IV [2,0 valores]

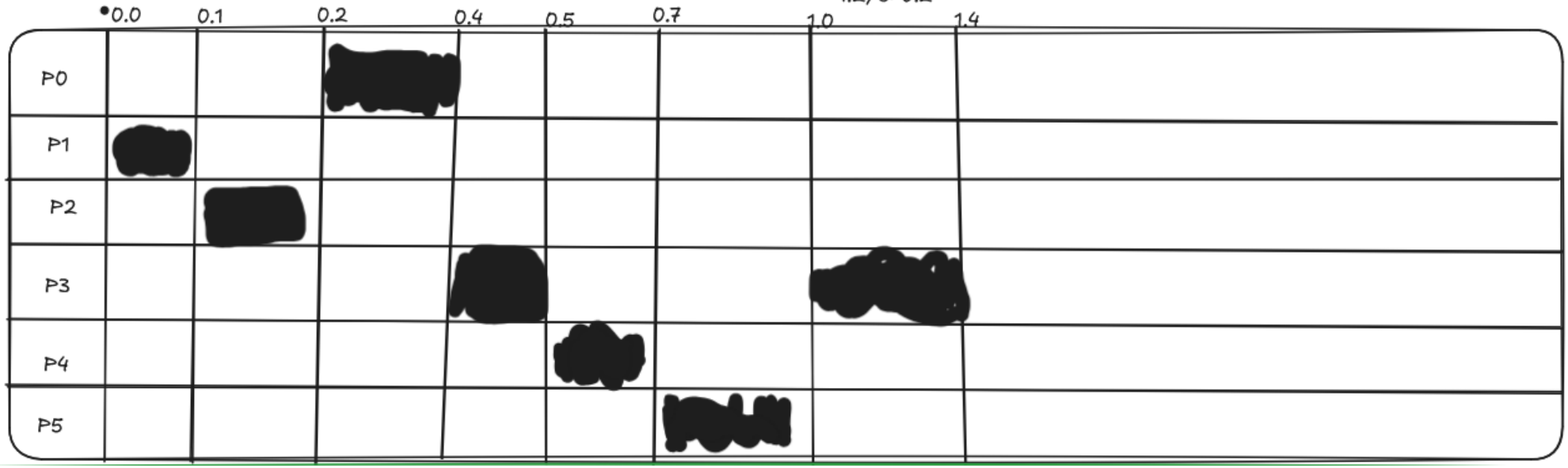
Considere o seguinte conjunto de processos. Instante de chegada e duração são indicados na tabela seguinte:

Processo	Instante de chegada	Duração
P0	0.0	0.2
P1	0.0	0.1
P2	0.1	0.1
P3	0.1	0.5
P4	0.5	0.2
P5	0.5	0.3

Calcule o tempo médio de espera, considerando que o algoritmo de escalonamento é o **SRTF (*Shortest Remaining Time First*)**. Fundamente a sua resposta com todos os cálculos que sentir necessidade de efetuar.

De seguida, comente se realizando os mesmos cálculos utilizando o algoritmo ***Round-Robin***, com um *quantum* de 0.6, a sequência de execução dos processos e o tempo de espera médio seriam diferentes do obtido com aplicação do algoritmo **SRTF**.

Para resolver este exercício, precisamos simular o algoritmo SRTF (Shortest Remaining Time First), que é a versão preemptiva do SJF (Shortest Job First). Isto significa que, a cada instante em que chega um novo processo ou um processo termina, o sistema escolhe executar o processo que tem o menor tempo restante para completar.



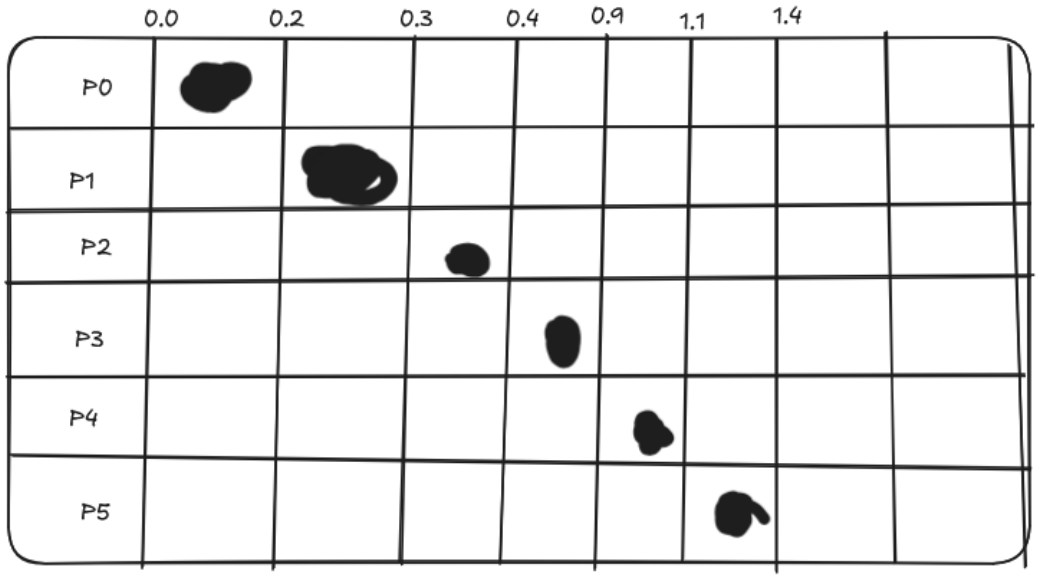
turnaround(FIM - CHEGADA)
P0 0.4 - 0.0 = 0.4 -->processo
P1 0.1 - 0.0 = 0.1
P2 0.2 - 0.1 = 0.1
P3 1.4 - 0.1 = 1.3
P4 0.7 - 0.5 = 0.2
P5 1.0 - 0.5 = 0.5

turnaround MEDIO : soma dos processos/n de processos
0.4+0.1+0.1+1.3+0.2+0.5=2.6
2.6/6 = 0.433

tempo de espera (turnaround - Duração)
P0 0.4 - 0.2 = 0.2 -->processo
P1 0.1 - 0.1 = 0.0
P2 0.1 - 0.1 = 0.0
P3 1.3 - 0.5 = 0.8
P4 0.2 - 0.2 = 0.0
P5 0.5 - 0.3 = 0.2

tempo de espera MEDIO: soma dos tempos /n de processos
0.2+0.0+0.0+0.8+0.0+0.2=1.2
1.2/6=0.2

De seguida, comente se realizando os mesmos cálculos utilizando o algoritmo ***Round-Robin***, com um *quantum* de 0.6, a sequência de execução dos processos e o tempo de ~~espera médio~~ *turnaround* seriam diferentes do obtido com aplicação do algoritmo **SRTF**.



TURNAROUND RR(FIM - CHEGADA)
P0 0.2 - 0.0 = 0.2 -->processo
P1 0.3 - 0.0=0.3
P2 0.4 - 0.1=0.3
P3 0.9 - 0.1=0.8
P4 1.1 - 0.5=0.6
P5 1.4 - 0.5=0.9

turnaround RR MEDIO : soma dos processos/n de processos
0.2+0.3+0.3+0.8+0.6+0.9=2.9
2.6/6 = 0.483

tempo de espera RR (turnaround - Duração)
P0 0.2 - 0.2 = 0.0 -->processo
P1 0.3 -0.1= 0.2
P2 0.3 -0.1= 0.2
P3 0.8 -0.5=0.3
P4 0.6 -0.2=0.4
P5 0.9 -0.3=0.6

tempo de espera RR MEDIO: soma dos tempos /n de processos
0.2+0.2+0.3+0.4+0.6+0.0=1.7
1.7/6=2.83

Em conclusão, o RR um quantum de 0,6 u.t. tem um desempenho pior (um turnaround maior) e a sequência de execução é diferente à da aplicação do SRTF.

Algoritmo FCFS (First-Come, First-Served) Os processos são executados pela ordem de chegada.

Algoritmo SJF (Shortest Job First) Os processos são executados por ordem crescente de duração.

Questão III [1,5 valores]

Assuma um sistema com **128 KB** de memória virtual por **paging**, com páginas de **16 KB**. Indique, recorrendo à técnica de *Memory Management Unit* (MMU) e à tabela seguinte:

110	1

P.PORTO ESCOLA SUPERIOR DE TECNOLOGIA E GESTÃO	Tipo de prova Teste Modelo	Ano letivo 2024/2025	Data
	Curso Licenciatura em Engenharia Informática Licenciatura em Segurança Informática e Redes de Computadores		Hora
	Unidade Curricular Sistemas Operativos		Duração

101	1
001	0
001	1
000	0
110	1

a) A que endereço físico (em decimal) corresponde o endereço virtual **25820** [0,75 valores]; 91356

b) A que endereço físico (em decimal) corresponde o endereço virtual **10000** [0,75 valores]. 108304

DADOS DO SISTEMA:

- Memória Virtual Total: 128 KB (2^17 bytes) -> Endereços de 17 bits
- Tamanho da Página: 16 KB (2^14 bytes) -> Offset de 14 bits
- Tamanho do VPN: 3 bits (17 - 14 = 3)

TABELA DE PÁGINAS (Índice -> PFN | Validade):

VPN 0 (000) -> Frame 6 (110) | Válido
VPN 1 (001) -> Frame 5 (101) | Válido <-- Usado na alínea a)
VPN 2 (010) -> Frame 1 (001) | Inválido
VPN 3 (011) -> Frame 1 (001) | Válido
VPN 4 (100) -> Frame 0 (000) | Inválido
VPN 5 (101) -> Frame 6 (110) | Válido

a) Converter Endereço Virtual: 25820
PASSO 1: Calcular VPN e Offset
- Cálculo: 25820 / 16384
- Inteiro (VPN): 1
- Resto (Offset): 9436

PASSO 2: Consultar Tabela
- Índice (VPN) 1 corresponde ao Frame Físico (PFN): 5 (Binário 101)
- Bit de validade é 1 (OK).

PASSO 3: Calcular Endereço Físico
- Fórmula: (PFN * Tamanho_Página) + Offset
- Cálculo: (5 * 16384) + 9436
- Cálculo: 81920 + 9436

RESULTADO FINAL A): 91356

To move canvas, hold Scroll wheel or Space while dragging, or use the hand tool

BANQUEIRO

Questão VI [2,0 valores]

Assuma um sistema com os tipos de recursos (A, B,...), processos (P1, P2,...) e caracterização como apresentada nas tabelas seguintes:

Alocado

	A	B	C
P0	0	0	2
P1	2	4	1
P2	3	0	2
P3	2	0	0

Necessidades máximas

	A	B	C
P0	4	3	3
P1	2	2	2
P2	9	0	2
P3	3	2	2

Disponibilidade

A	B	C	
2	3	1	

Aplique o algoritmo do banqueiro e determine se existe uma sequência de execução que mantenha o sistema num estado seguro. Apresente esta sequência juntamente com os cálculos que sentir necessidade de efetuar.

1. CÁLCULO DA MATRIZ DE NECESSIDADES (NEED)
Fórmula: Necessidade = Máximo - Alocado

Processo | Máximo (A,B,C) | Alocado (A,B,C) | Necessidade (A,B,C)

P0 | 4, 3, 3 | 0, 0, 2 | 4, 3, 1
P1 | 2, 2, 2 | 2, 4, 1 | 0, 0, 1 (*)
P2 | 9, 0, 2 | 3, 0, 2 | 6, 0, 0
P3 | 3, 2, 2 | 2, 0, 0 | 1, 2, 2

(*) Nota para P1: O recurso B alocado (4) é maior que o máximo (2).
Assumimos necessidade 0.
2. EXECUÇÃO PASSO A PASSO (SIMULAÇÃO)
Disponibilidade Inicial (Work): A=2, B=3, C=1
--- PASSO 1 ---
Verificar quem pode executar com Disponível (2, 3, 1):

- P0 precisa (4, 3, 1) -> Falso (precisa de 4 A, só tem 2)
- P1 precisa (0, 0, 1) -> Verdadeiro (0<=2, 0<=3, 1<=1)
- P2 precisa (6, 0, 0) -> Falso
- P3 precisa (1, 2, 2) -> Falso (precisa de 2 C, só tem 1)

> Decisão: Executar P1.
> P1 termina e devolve Alocados (2, 4, 1).
Nova Disponibilidade = (2, 3, 1) + (2, 4, 1) = (4, 7, 2)
Sequência segura até agora: [P1]

--- PASSO 2 ---
Verificar restantes com Disponível (4, 7, 2):
- P0 precisa (4, 3, 1) -> Verdadeiro
- P2 precisa (6, 0, 0) -> Falso (precisa de 6 A, só tem 4)
- P3 precisa (1, 2, 2) -> Verdadeiro

> Decisão: Executar P3 (Poderia ser P0 também).
> P3 termina e devolve Alocados (2, 0, 0).

Nova Disponibilidade = (4, 7, 2) + (2, 0, 0) = (6, 7, 2)
Sequência segura até agora: [P1, P3]

--- PASSO 3 ---
Verificar restantes com Disponível (6, 7, 2):
- P0 precisa (4, 3, 1) -> Verdadeiro
- P2 precisa (6, 0, 0) -> Verdadeiro

> Decisão: Executar P0.
> P0 termina e devolve Alocados (0, 0, 2).

Nova Disponibilidade = (6, 7, 2) + (0, 0, 2) = (6, 7, 4)
Sequência segura até agora: [P1, P3, P0]

--- PASSO 4 ---
Verificar restante com Disponível (6, 7, 4):
- P2 precisa (6, 0, 0) -> Verdadeiro

> Decisão: Executar P2.
> P2 termina e devolve Alocados (3, 0, 2).

Disponibilidade Final = (6, 7, 4) + (3, 0, 2) = (9, 7, 6)
Sim, existe uma sequência de execução que mantém o sistema num estado seguro.
Sequência segura até agora: [P1, P3, P0, P2]